

CS599 大作业报告

课程名 称	企业级应用软件设计与开发
项目名称	DeepResearch Agent: 融合 MCP_Visual 的多模态深度研究智能体系统
方向	Agentic AI 原生开发
学号	2025302935
姓名	赵天赐
专业	计算机技术 / 软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

1. 选题背景与设计思想
2. Specs 规格文档
3. 系统架构与设计
4. 关键实现与代码展示
5. 测试与评估
6. 系统升级与扩展
7. 课程总结
8. 附录：仓库与运行说明

一、选题背景与设计思想

1.1 问题定义

在课程学习、科研调研、技术选型和企业知识管理场景中，用户经常需要围绕一个复杂主题完成资料检索、信息筛选、交叉验证、观点综合和报告撰写。传统搜索引擎主要返回网页列表，用户仍需手工拆分问题、筛选来源、阅读材料并整理结论；通用聊天机器人虽然可以生成自然语言回答，但通常缺少可追踪的检索链路、稳定的工作流控制、可复用的工具系统和对上传材料的结构化处理能力。

本项目围绕“深度研究”这一典型 **Agentic AI** 任务，设计并实现 **DeepResearch Agent**。系统接收用户输入的研究主题，可选读取 **PDF**、图片或音频附件，自动完成研究意图拆解、多源搜索、内容过滤、综合分析、质量评估、报告撰写和知识库保存。与一次性问答不同，本项目强调“任务型智能体”的工程闭环：**Agent** 不只生成文本，还要按照可观测、可评估、可迭代的流程完成研究任务。

项目近期进一步融合 **MCP_Visual** 多模态能力。**PDF** 文档会被抽取为文本上下文，图片会经过视觉分析与 **OCR**，音频会经过转写和摘要，所有附件上下文都会进入研究规划、综合分析和最终报告撰写阶段。该设计使系统从“文本检索型 **Agent**”扩展为“多模态研究 **Agent**”，能够处理截图、图表、访谈录音、会议音频等更接近真实研究流程的输入。

1.2 现有方案不足

现有方案主要存在以下不足。

第一，搜索工具与生成模型之间缺少明确的任务分工。传统搜索引擎擅长召回资料，但无法自动判断研究范围是否充分，也无法将搜索结果组织成结构化报告；通用大模型擅长语言生成，但如果没有外部检索和来源约束，容易出现事实不稳定、引用缺失、论证链条不透明等问题。

第二，复杂研究任务需要多步骤控制。一个合格的研究报告通常包括问题拆解、资料获取、相关性判断、主题归纳、冲突识别、结论撰写和参考来源整理。单轮提示词难以稳定覆盖这些步骤，也难以在资料质量不足时自动补充检索。

第三，企业级应用要求工程化能力。真实系统需要 **API**、前端交互、数据库、任务状态、流式进度、错误处理、安全存储、测试用例和部署配置。仅有模型调用脚本无法满足课程对企业级应用软件设计与开发的要求。

第四，多模态资料处理能力不足。实际研究材料不只包含网页和 **PDF**，还包括图片、图表、屏幕截图、访谈录音和会议录音。没有多模态工具的研究系统无法充分利用这些输入，也难以体现 **Agent** 工具生态的扩展价值。

1.3 项目价值

DeepResearch Agent 的价值体现在三个层面。

在用户层面，系统降低了深度研究门槛。用户只需要输入研究主题并可选上传附件，系统即可自动完成资料检索、信息筛选和报告生成，并在前端实时展示进度、质量评分和参考来源。最终报告可以复制、下载或保存到知识库。

在技术层面，系统体现了 Agentic AI 原生开发思想。项目没有把大模型简单当作文本生成接口，而是围绕 LangGraph 构建状态机，将 LLM、搜索工具、PDF 解析、多模态分析、质量评估和报告生成组织为可扩展工作流。系统通过策略路由和质量阈值控制，使不同类型查询采用不同研究深度。

在工程层面，系统采用前后端分离、异步任务、SSE 流式推送、数据库持久化、向量检索和 Docker 基础设施，具备课程大作业所要求的完整软件系统形态。新增 MCP_Visual 后，项目也具备 MCP 协议和多模态 Agent 工具融合的分特征。

1.4 技术路线

项目技术路线可以概括为“前端任务入口 + 后端 Agent 编排 + 工具系统扩展 + 数据持久化 + 多模态上下文注入”。

前端使用 Next.js 和 React 构建交互界面，提供研究主题输入、附件上传、策略选择、研究进度展示、质量评分展示和报告操作。后端使用 FastAPI 提供 REST API 和 SSE 流式接口，使用 LangGraph 构建 Agent 工作流。数据库使用 PostgreSQL 存储任务、设置和知识库笔记，Chroma 用于知识库语义检索。工具层包括 Tavily/Web Search、Academic Search、Web Scraper、Calculator，以及新接入的 MCP_Visual 多模态工具。

核心思想是将复杂研究任务拆成多个可观测节点：Planning、Search、Evaluate Quality、Adaptive Control、Filter、Synthesis、Evaluate Coverage、Draft、Evaluate Report。每个节点只承担明确职责，节点之间通过状态对象传递数据。这样既便于调试，也便于后续替换模型、增加工具或调整策略。

二、Specs 规格文档

2.1 Product Spec

2.1.1 产品定位

DeepResearch Agent 是一个面向学习、科研调研和技术分析场景的深度研究智能体系统。系统面向需要快速形成结构化研究报告的用户，提供从问题输入到报告输出的一站式流程。

2.1.2 目标用户

目标用户包括课程学习者、科研初学者、技术调研人员和需要整理资料的企业知识工作者。用户通常具备明确主题，但缺少足够时间完成多轮检索和资料整理。

2.1.3 核心功能

- 1. 研究任务创建：用户输入研究主题，选择研究策略，可选上传 PDF、图片或音频附件。
- 2. API Key 配置：用户在设置页配置 LLM 提供商、模型和 API Key，后端加密保存。
- 3. 自动问题拆解：Agent 将复杂主题拆解为多个可检索子问题。
- 4. 多源检索：系统调用 Web Search、Academic Search 等工具获取资料。
- 5. 内容过滤：LLM 对搜索结果进行相关性筛选和摘要。
- 6. 质量评估：系统对搜索质量、覆盖度和报告质量进行评分。
- 7. 迭代优化：当质量不足时，系统可根据反馈优化查询并重新检索。
- 8. 多模态附件理解：系统读取 PDF 文本，分析图片内容和 OCR 文本，转写并摘要音频。
- 9. 报告生成：系统输出结构化 Markdown 研究报告，包含摘要、背景、深度分析、证据和参考来源。
- 10. 知识库保存：用户可将报告保存到知识库，并通过语义搜索复用。

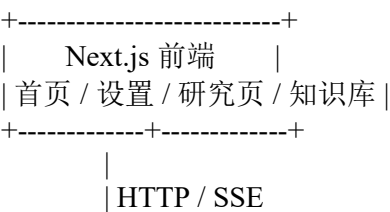
2.1.4 非功能需求

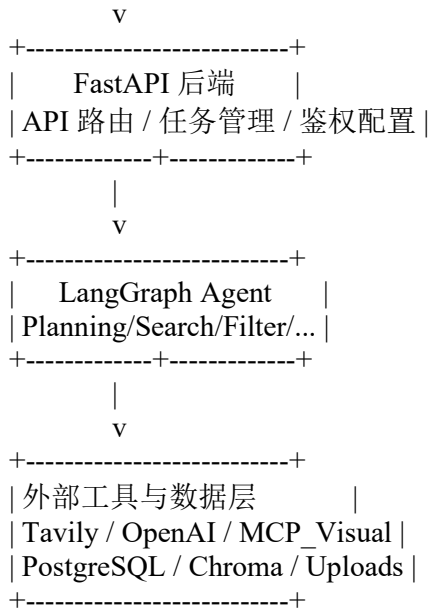
系统应满足以下非功能要求：接口响应清晰，研究过程可观测；API Key 不硬编码、不明文暴露；上传文件大小受限；研究任务失败时返回明确错误；前端在任务执行中保持可读进度；后端工具调用具有异常兜底；核心逻辑配有单元测试。

2.2 Architecture Spec

2.2.1 总体架构

系统采用前后端分离架构，核心由四层组成。





前端负责交互和展示，不直接接触数据库和外部搜索服务。后端负责 API、任务生命周期、配置管理和工具调度。**Agent** workflows 封装在后端内部，通过 **LangGraph** 的状态图编排节点。外部工具和数据层为 **Agent** 提供检索、生成、多模态理解和持久化能力。

2.2.2 Agent 状态设计

Agent 状态对象是节点间协作的核心数据结构，主要字段包括：

字段	含义
query	用户原始研究问题
uploaded_context / media_context	PDF、图片、音频附件解析后的上下文
sub_queries	Planning 生成的子查询
search_results	每个子查询对应的搜索结果
filtered_content	过滤后的关键内容
synthesis_text	综合分析文本
report	最终 Markdown 报告
sources	参考来源列表
quality_score	搜索质量评分
coverage_score	覆盖度评分
report_quality	报告质量评分
needs_refinement	是否需要优化查询
needs_more_research	是否需要补充研究

字段	含义
needs_revision	是否需要修改报告

该状态设计使每个节点可以专注于单一职责，并通过结构化字段向后续节点传递结果。

2.2.3 策略规格

系统提供四类研究策略：

策略	适用场景	节点特点
factual	快速事实查询	单轮规划、搜索、过滤、综合和起草
analytical	复杂分析任务	包含质量评估、覆盖度评估和报告评估
exploratory	开放式探索	子查询数量和搜索深度更高
comparative	对比分析	强调对比检索和报告质量评估

策略路由器可以根据用户显式选择或查询分类结果决定使用哪种策略。该机制避免所有任务都使用同一套固定流程，从而兼顾效率和深度。

2.3 API Spec

2.3.1 设置接口

GET /api/settings 用于读取用户设置，POST /api/settings 用于保存 LLM 提供商、模型和 API Key。API Key 通过后端安全模块加密后存储，前端不负责持久化密钥。

2.3.2 研究任务接口

POST /api/research/start 用于启动研究任务。请求体包括：

```
{
  "query": "研究主题",
  "deviceId": "设备标识",
  "fileId": "可选附件 ID",
  "strategy": "analytical"
}
```

响应体包括：

```
{
  "taskId": "任务 ID",
  "strategy": "analytical"
}
```

GET /api/research/{task_id}/stream 使用 SSE 返回任务进度事件。事件类型包括 step、classified、quality_update、sources、done 和 error。前端据此实时更新当前步骤、质量评分和完成状态。

GET /api/research/{task_id}/report 返回最终报告、来源、查询、策略和质量评分。

2.3.3 上传接口

POST /api/upload/pdf 保留旧版本兼容能力，仅支持 PDF。

POST /api/upload/media 支持 PDF、图片和音频，返回：

```
{
  "fileId": "上传文件 ID",
  "filename": "原始文件名",
  "size": 12345,
  "mediaType": "pdf | image | audio"
}
```

上传文件不会把原始文件名作为存储名，而是保存为随机 fileId + 扩展名，降低路径冲突和文件名注入风险。

2.3.4 知识库接口

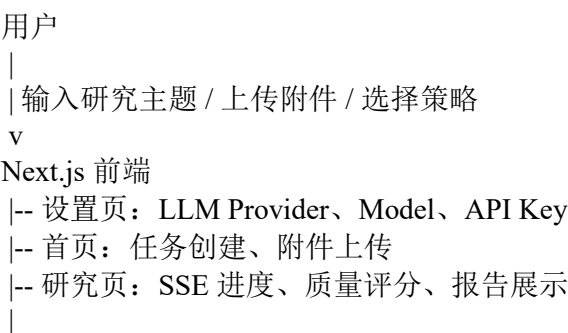
知识库接口包括笔记创建、查询、更新、删除和语义搜索。用户可以将研究报告保存为知识库笔记，并通过 Chroma 向量检索实现后续复用。

2.4 Security Spec

系统安全设计包括：API Key 加密存储，.env 不提交仓库；上传文件限制扩展名和大小；MCP_Visual 本地路径读取限定在允许目录；后端 CORS 限定本地前端和局域网开发地址；工具执行统一捕获异常并返回结构化错误；.gitignore 排除 .env、node_modules、虚拟环境、上传目录、日志和测试缓存。

三、系统架构与设计

3.1 核心架构图




```

v
FastAPI API 层
|-- /api/upload/media
|-- /api/research/start
|-- /api/research/{id}/stream
|-- /api/library
|
v
任务管理层
|-- 创建任务记录
|-- 启动异步研究任务
|-- 推送 SSE 事件
|
v
LangGraph Agent
|-- Planning
|-- Tool Search
|-- Evaluate Quality
|-- Adaptive Control
|-- Filter
|-- Synthesis
|-- Evaluate Coverage
|-- Draft
|-- Evaluate Report
|
v
工具与数据层
|-- Tavily / Web Search / Academic Search
|-- MCP_Visual: vision_analyze / ocr_image / audio_transcribe / audio_summarize
|-- PostgreSQL: 任务、设置、笔记
|-- Chroma: 语义检索
|-- Uploads: PDF、图片、音频文件

```

该架构将交互、业务编排、Agent 推理和数据工具分层处理。前端不关心 Agent 内部节点；Agent 不直接处理 HTTP；工具层不关心用户界面。分层结构有助于后续维护和扩展。

3.2 Agent 交互流程

```

Start
|
v
读取用户查询和附件上下文
|
v
Planning: 拆分子查询
|
v
Tool Search: 调用搜索工具获取资料

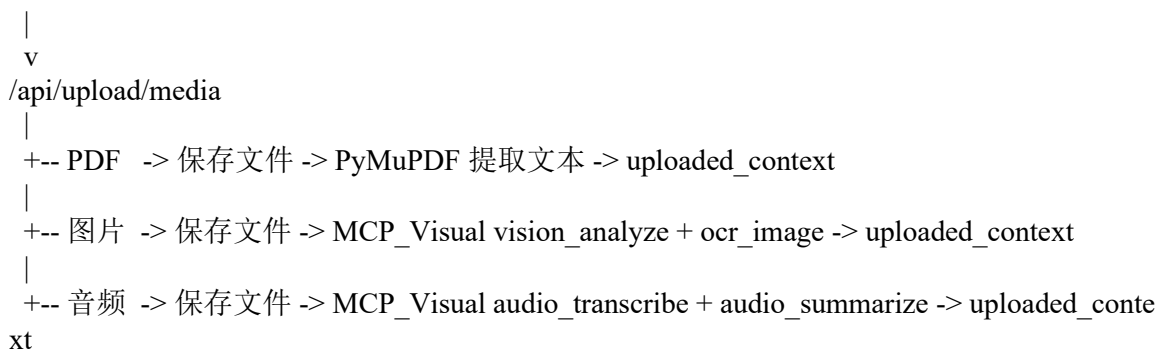
```



流程中的质量评估节点是本项目区别于普通聊天应用的关键。**Agent** 不默认第一轮结果一定可用，而是通过评分和阈值判断是否需要进一步优化查询、补充研究或修改报告。

3.3 多模态数据流设计

上传文件



```
|
v
Research Task State
|
+-- Planning 使用附件上下文生成更贴近材料的子查询
+-- Synthesis 将附件内容与搜索结果综合
+-- Draft 在最终报告中 使用附件证据
```

该设计没有把附件处理放在前端，也没有把多模态分析结果直接拼接到最终报告，而是在研究任务开始时将附件统一转化为 `uploaded_context`。这样可以保持 Agent 节点接口稳定，并允许 PDF、图片、音频共享同一上下文注入机制。

3.4 数据持久化设计

PostgreSQL 负责持久化用户设置、研究任务、步骤日志、报告和知识库笔记。研究任务执行过程中，任务状态会持续更新，包括当前步骤、进度、质量评分、覆盖度评分、报告质量和错误信息。当前端刷新页面或 SSE 连接中断时，可以通过任务状态和已完成任务回放机制恢复结果。

Chroma 用于知识库语义搜索。用户保存报告后，系统可以将报告内容嵌入向量库，后续通过语义查询检索历史笔记。这为 Agentic RAG 扩展提供了基础。

3.5 前端交互设计

前端包含首页、设置页、研究页和知识库页面。首页聚焦任务创建，避免复杂配置干扰用户；设置页管理模型和 API Key；研究页通过左侧步骤条展示 Agent workflow，通过质量评分面板展示搜索质量、覆盖度和报告质量；报告完成后提供复制、下载 Markdown 和保存到知识库操作。

这种设计让用户能够理解 Agent 正在做什么，而不是只等待一个黑盒结果。SSE 流式进度提高了长任务体验，也方便调试任务卡住或失败的位置。

四、关键实现与代码展示

4.1 LangGraph workflow 构建

系统使用 `build_research_graph` 根据策略动态构建 LangGraph 状态图。节点序列由策略对象提供，路由规则根据 `needs_refinement`、`needs_more_research`、`needs_revision` 和质量阈值决定下一步。

核心逻辑示意：

```
def build_research_graph(strategy: ResearchStrategy | None = None):
    graph = StateGraph(dict)
    node_sequence = strategy.get_node_sequence()
    routing_rules = strategy.get_routing_rules()
```

```

for node_name in all_node_names:
    graph.add_node(node_name, _make_node_wrapper(node_name))

graph.set_entry_point(node_sequence[0])
graph.add_conditional_edges(node_name, router)
return graph.compile()

```

该实现的价值在于：同一套节点可以被不同策略组合复用，新增策略不需要重写全部业务逻辑，只需声明节点顺序和路由条件。

4.2 策略路由与研究深度控制

系统定义了 `factual`、`analytical`、`exploratory` 和 `comparative` 四种策略。不同策略设置不同的最大迭代次数、子查询数量、搜索深度和质量阈值。例如，事实查询使用较少子查询并减少迭代，深度分析任务则包含质量评估、覆盖度评估和报告评估。

自适应控制器根据查询类别和复杂度计算初始研究深度，并在迭代过程中判断是否继续。若质量和覆盖度均达标，则停止额外检索；若连续迭代收益很低，则避免无意义消耗。

4.3 工具系统

工具层抽象为 `BaseTool` 和 `ToolResult`。每个工具提供 `execute` 方法和 `OpenAI function schema`。`ToolExecutor` 根据工具注册表加载工具，将工具 `schema` 提供给 LLM，并解析 LLM 生成的 JSON 工具调用请求。

工具执行示意：

```

class BaseTool(ABC):
    name: str = ""
    description: str = ""

    @abstractmethod
    async def execute(self, **kwargs) -> ToolResult:
        ...

    def to_openai_function(self) -> dict:
        return {
            "type": "function",
            "function": {
                "name": self.name,
                "description": self.description,
                "parameters": self.get_parameters(),
            },
        }

```

当前工具包括 Web Search、Academic Search、Web Scraper、Calculator 和 MCP_Visual 多模态工具。工具注册制使后续添加新能力的成本较低。

4.4 MCP_Visual 融合实现

新增多模态服务 MultimodalAnalysisService 负责统一处理附件上下文。其设计重点包括：

1. 从 fileId 找到后端上传目录中的真实文件路径。
2. 判断媒体类型：PDF、图片或音频。
3. PDF 使用 PyMuPDF 提取文本。
4. 图片调用 MCP_Visual 的 MediaResolver 和 OpenAIProvider，完成视觉分析和 OCR。
5. 音频调用 MCP_Visual 完成转写和摘要。
6. 将多模态结果格式化为统一的 uploaded_context。
7. 如果没有可用 OpenAI key 或分析失败，记录警告并跳过多模态上下文，不中断研究任务。

上下文注入逻辑示意：

```
uploaded_context = await multimodal_analysis_service.build_uploaded_context_async(
    file_id,
    api_key=api_key,
    provider=provider,
)

state = {
    "query": query,
    "uploaded_context": uploaded_context,
    "media_context": uploaded_context,
    "sub_queries": [],
    "search_results": {},
}
```

该实现让多模态能力成为 Agent 工作流的一部分，而不是独立的文件解析功能。附件内容会影响问题拆解、综合分析和报告撰写。

4.5 上传接口实现

系统新增 /api/upload/media，统一支持 PDF、图片和音频。接口检查文件名、扩展名和大小，保存文件后返回 fileId 和 mediaType。旧的 /api/upload/pdf 保留，以保证前后兼容。

```

@router.post("/media")
async def upload_media(file: UploadFile = File(...)) -> dict:
    if suffix not in SUPPORTED_MEDIA_EXTENSIONS:
        raise HTTPException(status_code=400, detail="Only PDF, image, and audio files are supported")

    content = await file.read()
    if len(content) > settings.max_file_size_mb * 1024 * 1024:
        raise HTTPException(status_code=400, detail="File size exceeds limit")

    file_id, media_type = pdf_service.save_media_file(content, file.filename)
    return {"fileId": file_id, "filename": file.filename, "size": len(content), "mediaType": media_type}

```

4.6 前端关键实现

首页上传控件接受 .pdf,image/*,audio/*, 统一调用 /api/upload/media。上传成功后仍沿用 fileId 字段启动研究任务，避免数据库迁移和大范围接口变更。

```

const uploadRes = await fetch(`${API_BASE_URL}/api/upload/media`, {
  method: "POST",
  body: formData,
});

```

```

const uploadData = await uploadRes.json();
fileId = uploadData.fileId;

```

研究页通过 SSE 接收进度事件，并展示当前步骤、策略、迭代轮次、搜索质量、覆盖度和报告质量。报告完成后，用户可以复制全文、下载 Markdown 或保存到知识库。

4.7 配置文件与安全实现

系统通过 .env.example 明确列出数据库、Chroma、Tavily、CORS 和 MCP_Visual 相关配置。真实 .env 被 .gitignore 排除，避免泄露密钥。API Key 使用后端加密模块处理，前端只负责输入和提交。

MCP_Visual 相关变量包括：

```

OPENAI_API_KEY
MCP_VISUAL_ALLOWED_DIRS
MCP_VISUAL_MAX_FILE_MB
MCP_VISUAL_MAX_DOWNLOAD_MB
MCP_VISUAL_REQUEST_TIMEOUT_SECONDS
OPENAI_VISION_MODEL
OPENAI_SUMMARY_MODEL
OPENAI_TRANSCRIBE_MODEL
OPENAI_DIARIZE_MODEL

```

4.8 AI IDE 使用说明

本项目开发过程中以 AI IDE 辅助完成代码阅读、方案制定、增量实现、测试修复和文档撰写。AI IDE 在本项目中的作用不是替代工程判断，而是辅助快速理解项目结构、生成候选实现、检查遗漏和补充测试。关键决策仍围绕课程要求和代码实际结构展开，例如保留 `ResearchStart.fileId` 避免数据库迁移、采用内嵌 `MCP_Visual` 降低部署复杂度、在多模态失败时不中断研究任务等。

建议在最终提交报告时补充 Trae CN 或实际 AI IDE 的使用截图，包括：代码补全/对话生成、Agent 工具集成修改、测试运行结果和 README 修改记录。

五、测试与评估

5.1 功能测试

项目针对核心模块设置了单元测试和集成测试，覆盖自适应控制、查询分类、工具执行、节点逻辑、策略定义、安全逻辑和多模态集成。

已验证的后端测试结果：

```
pytest backend/tests
85 passed
```

已验证的 MCP_Visual 测试结果：

```
pytest MCP_Visual/tests
10 passed
```

已验证的前端静态检查结果：

```
npm run lint
No ESLint warnings or errors
```

5.2 多模态集成测试

新增测试 `test_multimodal_integration.py` 覆盖以下场景：

1. `/api/upload/media` 接受图片上传并返回 `mediaType=image`。
2. 不支持的文件类型会返回 400 错误。
3. 使用 fake MCP provider 验证图片分析结果会被格式化进 `uploaded_context`。
4. ToolExecutor 初始化后可以发现 `vision_analyze`、`ocr_image`、`audio_transcribe`、`audio_summarize` 和 `media_qa`。
5. MCP_Visual 工具包装器可以返回结构化 ToolResult。

这些测试避免真实调用 OpenAI，重点验证系统集成路径和数据结构是否正确。

5.3 Agent 行为评估

Agent 行为可以从以下维度评估：

评估维度	评价方式	当前实现
任务拆解能力	子查询是否覆盖主题关键方面	Planning 节点生成 3-5 个子查询
检索质量	搜索结果是否相关、来源是否可用	Evaluate Quality 节点评分
覆盖充分性	综合分析是否覆盖原始问题	Evaluate Coverage 节点评分
报告质量	报告结构、事实性、引用是否合格	Evaluate Report 节点评分
工具使用能力	是否能按需调用搜索/多模态工具	ToolExecutor + 工具注册表
可观测性	用户是否能看到执行进度	SSE 实时事件和前端步骤条

当前系统已具备基础评估闭环，但仍缺少大规模 benchmark。后续可以构建固定题库，从准确性、覆盖度、引用质量、耗时和 token 成本等维度进行量化比较。

5.4 Demo 场景设计

建议最终展示使用以下 Demo 场景。

场景一：文本主题研究。输入“AI Agent 在企业知识管理中的应用现状与风险”，选择深度分析策略，观察 Agent 拆解子查询、检索来源、生成报告并保存到知识库。

场景二：PDF 辅助研究。上传一份课程材料或论文 PDF，输入“基于这份材料总结系统设计方法”，观察系统将 PDF 文本注入 Planning 和 Draft。

场景三：图片辅助研究。上传一张系统架构图或数据图表截图，输入“请基于图片内容分析系统模块关系”，观察 MCP_Visual 的视觉摘要和 OCR 结果进入报告。

场景四：音频辅助研究。上传访谈或课堂录音片段，输入“根据音频内容提炼关键观点并补充外部资料”，观察音频转写和摘要对研究结果的影响。

5.5 当前限制

项目仍存在一些限制。第一，多模态分析依赖 OpenAI 视觉和音频模型，如果没有可用 OpenAI key，只能跳过图片和音频上下文。第二，当前版本一次研究任务只支持一个附件。第三，Web 搜索质量依赖外部 Tavily 服务和网页可访问性。第四，Agent 行为评估目前以节点评分和单元测试为主，缺少统一 benchmark 数据集。第五，生产部署还需要进一步完善鉴权、限流、任务队列和监报告警。

六、系统升级与扩展

6.1 可扩展架构

系统采用注册式工具、策略式工作流和状态机编排，具有较好的扩展性。新增工具时，只需继承 `BaseTool` 并注册到工具表；新增研究策略时，只需声明节点序列和路由规则；新增附件类型时，可以在上传服务和多模态服务中增加解析逻辑，再统一转化为 `uploaded_context`。

6.2 下一阶段计划

下一阶段可以从以下方向升级。

第一，支持多附件研究。当前任务只支持一个 `fileId`，后续可以将 `ResearchStart.fileId` 扩展为 `fileIds`，允许用户同时上传 PDF、图片和音频，并为每个附件生成独立上下文块。

第二，引入 **Agentic RAG**。系统已经具备知识库和 **Chroma**，可以进一步让 **Agent** 在研究过程中主动检索历史报告和笔记，将个人知识库与外部搜索结合。

第三，增强引用追踪。当前报告会列出来源 **URL**，后续可以把每段结论绑定到具体来源片段，提高可验证性。

第四，完善 **benchmark**。构建固定测试集，评价报告事实准确性、覆盖度、引用质量、生成耗时和成本。

第五，云端部署。可将前端部署到 **Vercel** 或 **Netlify**，后端部署到 **Render**、**Railway** 或云服务器，数据库使用托管 **PostgreSQL**，并提供课程评阅可访问 **URL**。

第六，生产级运维。引入后台任务队列、**Redis** 队列、结构化日志、请求追踪、错误监控和限流策略，提高稳定性和可观测性。

6.3 AI 能力演进路径

系统的 **AI** 能力可以沿以下路径演进：

1. 从单轮生成到多节点 **Agent** 工作流。
2. 从文本检索到多模态上下文理解。
3. 从外部搜索到个人知识库 **RAG**。
4. 从固定策略到基于任务难度的动态规划。
5. 从自然语言报告到可追溯证据链和结构化知识沉淀。

该路径符合 **Agentic AI** 应用从原型到工程系统的发展规律。

七、课程总结

7.1 个人收获

通过本项目，我对企业级应用软件设计与开发有了更完整的认识。一个 AI 应用并不等于一次模型调用，真正可用的系统需要清晰的产品定位、稳定的 API、可靠的数据流、可观测的任务状态、可维护的代码结构和可验证的测试体系。

在 Agentic AI 方面，我理解到智能体系统的关键不只是提示词，而是 workflow 设计。Planning、Search、Filter、Synthesis、Evaluation 和 Draft 等节点将复杂任务拆成可控步骤，使系统具备调试和迭代能力。质量评估和覆盖度评估也让我认识到，Agent 需要自我检查机制，否则很难稳定完成复杂任务。

在工程实践方面，我进一步熟悉了 FastAPI、Next.js、LangGraph、Prisma、PostgreSQL、Chroma、Docker 和 SSE 等技术的协同方式。尤其是 MCP_Visual 的接入过程体现了工程权衡：为了降低部署复杂度，第一版采用内嵌调用；为了避免大范围迁移，保留 fileId 单附件接口；为了保证任务稳定，多模态分析失败时不让研究任务整体失败。

7.2 工程思维转变

本项目让我从“实现功能”转向“设计系统”。在早期开发中，容易只关注某个接口或某个页面是否可用；但课程项目要求从规格、架构、实现、测试、部署和文档多个角度审视系统。一个工程系统必须考虑长期维护：模块边界是否清楚、异常是否可处理、配置是否安全、测试是否覆盖关键路径、文档是否能让他人运行项目。

我也认识到 AI IDE 的正确使用方式。AI 可以帮助快速阅读代码、生成样例、发现遗漏和补充测试，但不能替代对系统边界和需求优先级的判断。开发者仍需要决定哪些功能应该做、哪些风险必须控制、哪些实现会影响后续扩展。

7.3 对课程的建议

建议课程继续强调 Specs、架构图和工程实现之间的一致性。对于 AI 应用课程项目，可以进一步提供 Agent 评估样例，例如固定 benchmark、引用质量评分标准、工具调用日志分析方法等，帮助学生从“能运行”提升到“可评估”。同时，建议保留 MCP、RAG、云部署等加分项，因为这些内容能引导学生接触真实 AI 应用系统所需的前沿工程能力。